PROBLEMA:	**OAT 2 - Padrão de Projeto e Diagrama de Atividade**	**DATA:**	**04/09/2026**
PILOTO (A):	Murillo dos Santos Marinho Ferreira	**COPILOTO (A):**	Jadson da Silva Sobrinho
QA:	Jadson da Silva Sobrinho	**ARQUITETO (A):**	Levi Falcão de Queiroz
SCRUM MASTER:	Mateus Queiroz Matos Ferreira		

Objetivo: Modelar as novas entidades (OS e Pedido de Peças), criar a Entidade Associativa e estruturar a visão lógica através do Diagrama de Atividade.

Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 2 e o Arquiteto documenta no repositório):

- **Status Obrigatórios:** De acordo com as Regras de Negócio da OAT 2, quais são os status exatos e obrigatórios que devem compor a Ordem de Serviço (OS) e o Pedido de Peças?
    - ***Fato Extraído:*** *Status - Aberto, Pendente de Pagamento, Pago, Em Execução, Executado (enum)*
- **Relacionamento de Peças e Pedidos:** O que a OAT 2 exige que seja criado para relacionar as Peças aos Pedidos, gerenciando corretamente as quantidades?
    - ***Fato Extraído:*** *Status - Orçando, Pendente de Pagamento, Pago/Faturado, Entregue (enum)*

Brainstorm Investigativo (Questões e Ideias):

- **Diagrama de Atividade:** A OAT exige a implementação visual de 1 (um) endpoint focando na visão do controller. Qual endpoint a equipe escolherá? Quais os caminhos de sucesso e de erro (ex: validação falha) desse fluxo?
  O endpoint de criação de Ordem de Serviço, porque ele permite demonstrar praticamente todos os requisitos novos. O que pode dar erro talvez sejam dados invalidos ou url incorreta.

- **Lombok:** O que a diretoria definiu sobre o uso da biblioteca Lombok na restrição tecnológica? Como isso afeta o código de hoje?
  “É expressamente proibido o uso da biblioteca Lombok neste desenvolvimento”. Com o uso de Lombok, poderia aumentar o tempo de codificação, porém abrindo mão do entendimento profundo de como funciona.

Cenários de Teste (Liderado pelo QA):

- Validar (visualmente ou no código) se é possível adicionar a mesma peça múltiplas vezes a um pedido através da entidade associativa.

Entrega do Encontro:

- **Artefatos:** O Arquiteto desenha a primeira versão do Diagrama de Atividade do endpoint escolhido.

- **Código:** Criação das classes `OrdemServico.java`, `PedidoPecas.java` e da Entidade Associativa no repositório, com getters e setters manuais.

PROBLEMA:	**OAT 2 - Padrão de Projeto e Diagrama de Atividade**	**DATA:**	**11/09/2026**
PILOTO (A):	Mateus Queiroz Matos Ferreira	**COPILOTO (A):**	Murillo dos Santos Marinho Ferreira
QA:	Jadson da Silva Sobrinho	**ARQUITETO (A):**	Jadson da Silva Sobrinho
SCRUM MASTER:	Levi Falcão de Queiroz		

Objetivo: Lidar com a complexidade de instanciar objetos ricos implementando o Padrão de Projeto *Builder* na construção da Ordem de Serviço.

Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 2 e o Arquiteto documenta no repositório):

- **Obrigatoriedade Criacional:** Segundo a seção de Padrões de Projeto, qual padrão deve obrigatoriamente ser utilizado na construção de uma Ordem de Serviço (OS)?
    - *Fato Extraído: Builder*
- **A Persistência:** Houve alguma alteração nos *repositories* em relação à atividade passada?
    - *Fato Extraído: Os repositories passaram a seguir o padrão Singleton, ou seja, existe uma única instância do repository durante a execução da aplicação.*

Brainstorm Investigativo (Questões e Ideias):

- **A Anatomia do Builder:** Como implementar o Builder manualmente sem o Lombok? Onde fica a classe estática aninhada? Como os métodos retornam a própria instância (this) para permitir o encadeamento?

  Criando uma classe estática aninhada Builder dentro da própria classe OrdemServico. A classe estatica aninhada fica dentro da classe OrdemServico. Ela é static porque podemos criar um Builder sem precisar criar primeiro uma OrdemServico e o this representa a própria instância atual do Builder.

- **O Construtor Privado:** Por que o construtor da classe OrdemServico passa a ser privado e recebe apenas o objeto Builder como parâmetro?

  Porque essa estrutura faz parte da própria ideia do Builder: a criação da OrdemServico passa a ser controlada pelo Builder, em vez de permitir que qualquer parte do sistema monte o objeto diretamente.

Cenários de Teste (Liderado pelo QA):

- O código deve falhar (em tempo de compilação) se o Desenvolvedor Piloto tentar criar uma OS utilizando o operador new OrdemServico() em vez de passar pelo OrdemServico.builder().

Entrega do Encontro:

- **Código:** Classe OrdemServico refatorada e completamente adaptada para ser construída utilizando o Padrão *Builder*.
- **Diagrama:** Arquiteto atualiza o Diagrama de Classes para representar a estrutura do Builder (classe aninhada e relacionamento).

PROBLEMA:	**OAT 2 - Padrão de Projeto e Diagrama de Atividade**	**DATA:**	**18/09/2026**
PILOTO (A):	Levi Falcão de Queiroz	**COPILOTO (A):**	Mateus Queiroz Matos Ferreira
QA:	Murillo dos Santos Marinho Ferreira	**ARQUITETO (A):**	Jadson da Silva Sobrinho
SCRUM MASTER:	Jadson da Silva Sobrinho		

Objetivo: Isolar o domínio da aplicação aplicando o padrão DTO (Data Transfer Object) e construindo o roteamento (User Stories) com foco seguro.

Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 2 e o Arquiteto documenta no repositório):

- **Separação de Contexto:** A OAT 2 permite que os *Controllers* recebam e devolvam entidades de domínio puro? Qual a regra em relação aos DTOs e Mappers?
    - *Fato Extraído: Os Controllers não devem receber nem devolver diretamente entidades de domínio. O correto é: Controller <> DTO <> Mapper <> Entidade, ou seja, Controller recebe/devolve DTO; Mapper faz a conversão; Repository trabalha com as entidades.*

Brainstorm Investigativo (Questões e Ideias):

- **A Conversão:** Como os métodos Mappers farão o de-para (da conversão do OrdemServicoDTO para OrdemServico)? Faremos de forma estática?
- sim, faremos o de-para de maneira estática: toEntity() para DTO > entidade e toDTO() para entidade > DTO. O Mapper não armazena dados; apenas realiza a conversão.

- **O Roteamento:** Para atender as US01 (Criar OS), US03 (Criar Pedido) e afins, quais verbos HTTP (@PostMapping, @PutMapping) e status de sucesso mapearemos?

User Story / Funcionalidade	Operação	Endpoint	HTTP	Status de sucesso
US01 – Criar OS	Criar uma Ordem de Serviço	`/api/ordens-servico`	POST	**201 Created**
Consultar OS	Listar todas as OS	`/api/ordens-servico`	GET	**200 OK**
Consultar OS	Buscar OS por código	`/api/ordens-servico/{codigo}`	GET	**200 OK**
Atualizar OS	Alterar uma OS	`/api/ordens-servico/{codigo}`	PUT	**200 OK**
Excluir OS	Excluir uma OS	`/api/ordens-servico/{codigo}`	DELETE	**204 No Content**
US03 – Criar Pedido	Criar Pedido de Peças	`/api/pedidos-pecas`	POST	**201 Created**
Consultar Pedido	Listar pedidos	`/api/pedidos-pecas`	GET	**200 OK**
Consultar Pedido	Buscar pedido por código	`/api/pedidos-pecas/{codigo}`	GET	**200 OK**
Adicionar peças	Adicionar peça a pedido existente	`/api/pedidos-pecas/{codigo}/itens`	POST	**201 Created**
Modificar status	Alterar status do pedido	`/api/pedidos-pecas/{codigo}/status`	PUT	**200 OK**
Remover peça	Remover item do pedido	`/api/pedidos-pecas/{codigo}/itens/{itemCodigo}`	DELETE	**204 No Content**

Cenários de Teste (Liderado pelo QA):

- O QA mapeia os campos enviados em um JSON de requisição; O sistema não deve quebrar caso o cliente não envie atributos internos da entidade, já que o DTO restringe a carga útil.

Entrega do Encontro:

- **Código:** Classes de DTO (OrdemServicoDTO, etc.) e Mappers finalizadas. Implementação dos Controllers da OS e do Pedido.
- **Artefatos:** *Collection* (Postman ou Insomnia) de testes devidamente populada e exportada no repositório. O Arquiteto faz os ajustes finais no Diagrama de Classes adicionando os DTOs.